

Intelligence Leading the Frontier, Computing Shaping the Future HIT Code Review Research

Experiment Report

Mining Code Review Comments and Understanding Requirements

Name	<u>Ulugbek Khamidov, Komil Tokhirov</u>	Department	<u>HITCodeReviewResearch</u>
Student ID	<u>2026140348, 2026140354</u>	Course Teacher	Liu Jiakun
Instructor	Liu Jiakun	Lab Location	<u>HIT First Campus</u>
Lab Time	<u>2024-06-11 14:00-17:00</u>		
Lab Score		Discussion Score	
Report Score		Total Experi- ment Score	

1. Experiment Objectives

- Understood PR and code review workflows in modern development. - Learned GitHub repository code review data structure. - Mastered relationships between PRs, commits, reviews, and comments. - Retrieved code review data via GitHub API. - Extracted code, text, and metadata from PRs. - Applied statistical analysis to build dataset for subsequent experiments.

2. Experiment Content

- Selected 5 projects: pytorch, tensorflow, kubernetes, vscode, langchain. - Retrieved 300 PRs per project (1,550 total after trimming). - Extracted PR metadata: ID, title, body, author, creation time, merge status. - Retrieved reviews: reviewer, decision, time, comment text. - Extracted commits: messages and diff content. - Collected labels from

each PR. - Detected AI reviewers using keyword matching. - Detected AI-generated code using keyword matching. - Performed statistical analysis on the full dataset.

3. Experimental Results

We collected 1,550 PRs from 5 open-source projects. Key statistics: - 813 merged (52.5%), 737 unmerged (47.5%). - Average additions: 9,110 lines. - Average deletions: 428 lines. - Average changed files: 88. - Average reviewers per PR: 1.8. - Average review comments per PR: 1.6. - AI reviewers present in 471 PRs (30.4%). - AI-generated code detected in 530 PRs (34.2%).

Six visualizations were produced showing merge status, review comments distribution, reviewers per PR, PR length, AI vs human code comparison, and top 10 labels.

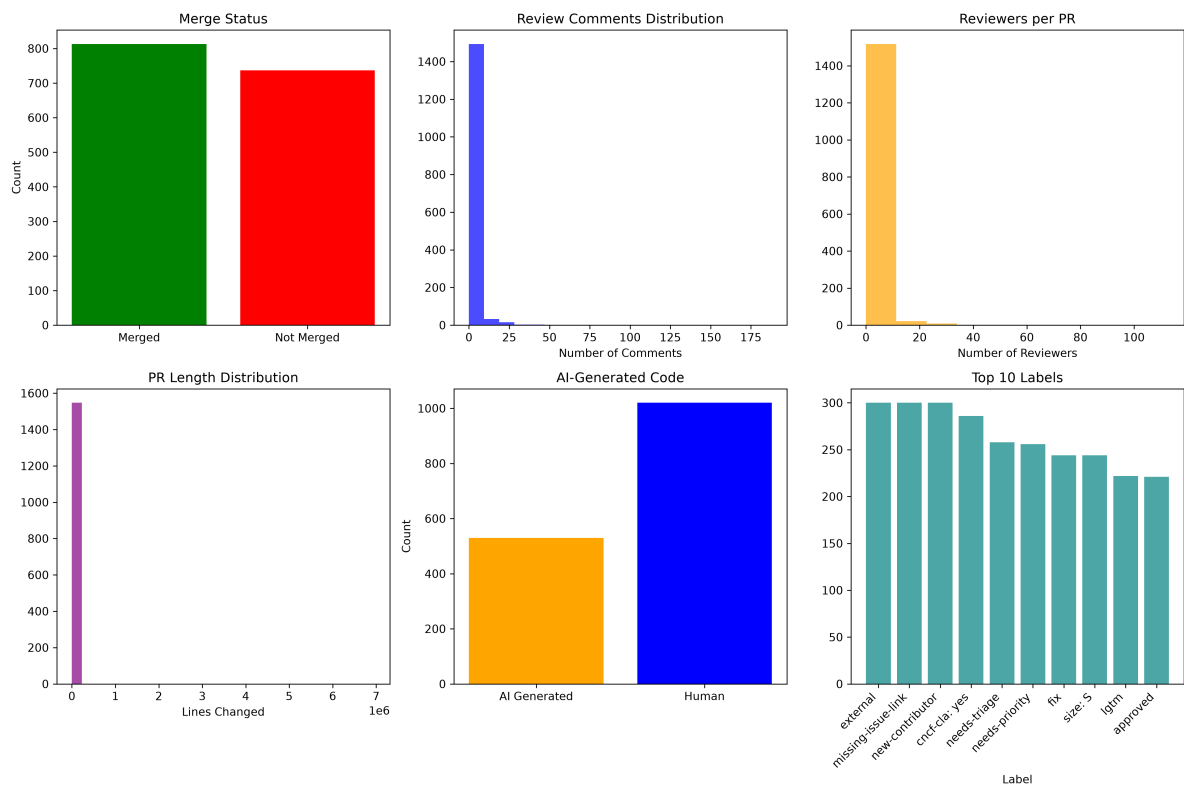


Figure 1: Statistical analysis of the PR dataset

4. Summary of Problems Encountered

- API rate limiting: added 0.15s delays between requests. - Network interruptions: implemented checkpoint saving every 5 PRs. - Langchain contained 9,450 PRs. Trimmed to 300 most recent for balanced distribution. - PyGithub deprecation warnings. Code functions correctly despite warnings.

5. Reflections on the Experiment

- PR is a merge request containing discussion and multiple commits. A commit is a single code change. - Code review catches defects early, enforces standards, and spreads knowledge across the team. - Merge prediction can flag high-risk PRs, prioritize reviews, and automate CI decisions. - AI reviewers respond instantly but lack project-specific context. Human reviewers understand broader design implications. - Our dataset has moderate imbalance (52.5% merged). SMOTE or class weighting can address this.

Instructor's Comments: Date:

Intelligence Leading the Frontier, Computing Shaping the Future HIT Code Review Research

Experiment Report Mining Code Review Comments and Understanding Requirements

Name	Ulugbek Khamidov, Komil Tokhirov	Department	HITCodeReviewResearch
Student ID	2026140348, 2026140354	Course Teacher	Liu Jiakun
Instructor	Liu Jiakun	Lab Location	HIT First Campus
Lab Time	2024-06-11 14:00-17:00		
Lab Score		Discussion Score	
Report Score		Total Experi- ment Score	

1. Experiment Objectives

- Understood traditional machine learning workflow for code review. - Mastered data preprocessing for code review tasks. - Extracted AST and CFG representations from code. - Extracted structural, textual, and statistical features. - Trained SVM and Random Forest models. - Built a merge prediction model and analyzed performance.

2. Experiment Content

- Filtered 1,020 human-written PRs from Experiment 1 dataset. - Extracted AST representations using Python's ast module. - Extracted CFG representations for control flow analysis. - Extracted 9 features: additions, deletions, changed_files, commits_count, reviewers_count, comments_count, labels_count, pr_length, has_ai_reviewer. - Constructed ML dataset: 816 train, 204 test. - Trained SVM (RBF kernel) and Random Forest (100 trees). - Evaluated using accuracy, precision, recall, F1, ROC-AUC.

3. Experimental Results

We trained SVM and Random Forest on 1,020 human-written PRs (601 merged, 419 not merged).

SVM Results: - Accuracy: 88.7% - Precision: 88.2% - Recall: 93.3% - F1: 90.7% - ROC-AUC: 92.3%

Random Forest Results: - Accuracy: 91.2% - Precision: 88.6% - Recall: 97.5% - F1: 92.9% - ROC-AUC: 97.3%

Random Forest outperformed SVM across all metrics. Feature importance analysis showed labels_count (45.3%) as the dominant predictor, followed by pr_length (12.2%) and deletions (10.3%).

AST and CFG examples were generated for sample code snippets.

Figures

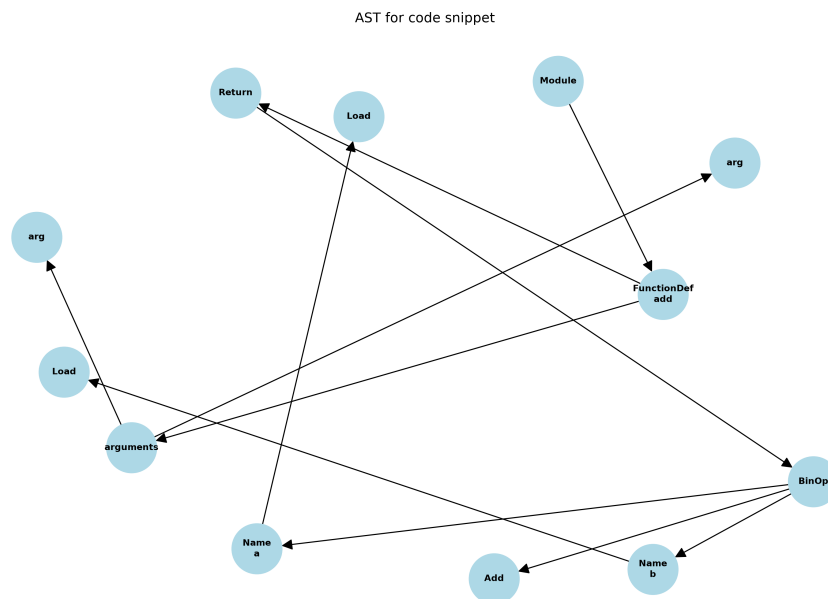


Figure 1: AST for a simple addition function

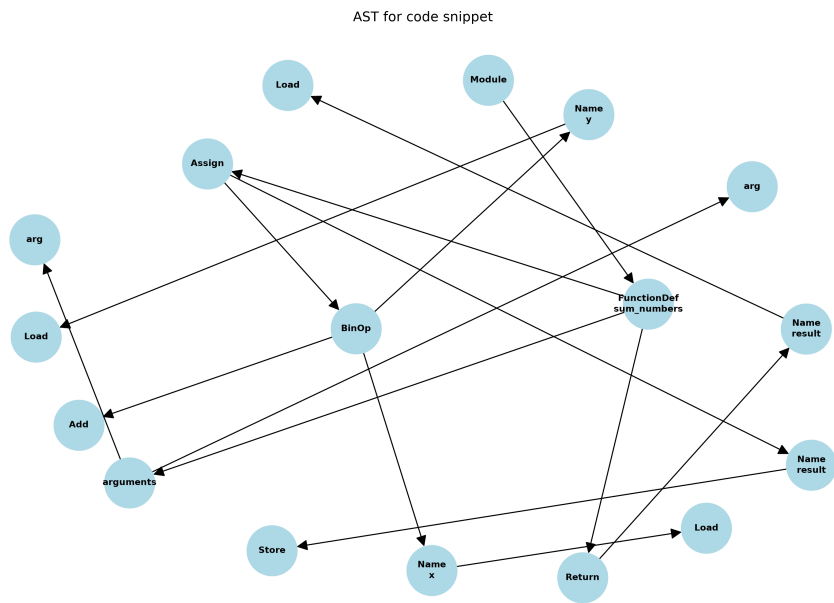


Figure 2: AST for a function with intermediate variable

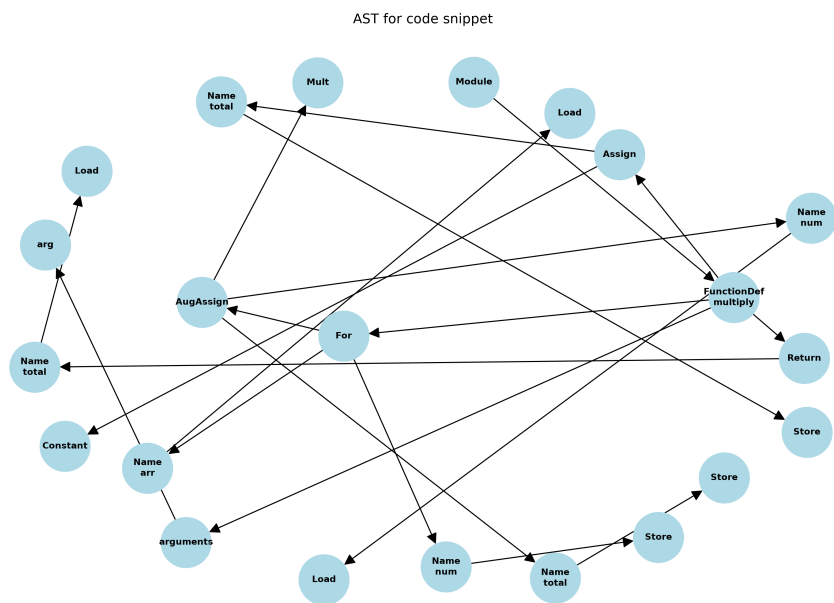


Figure 3: AST for a function with loop

CFG for code snippet

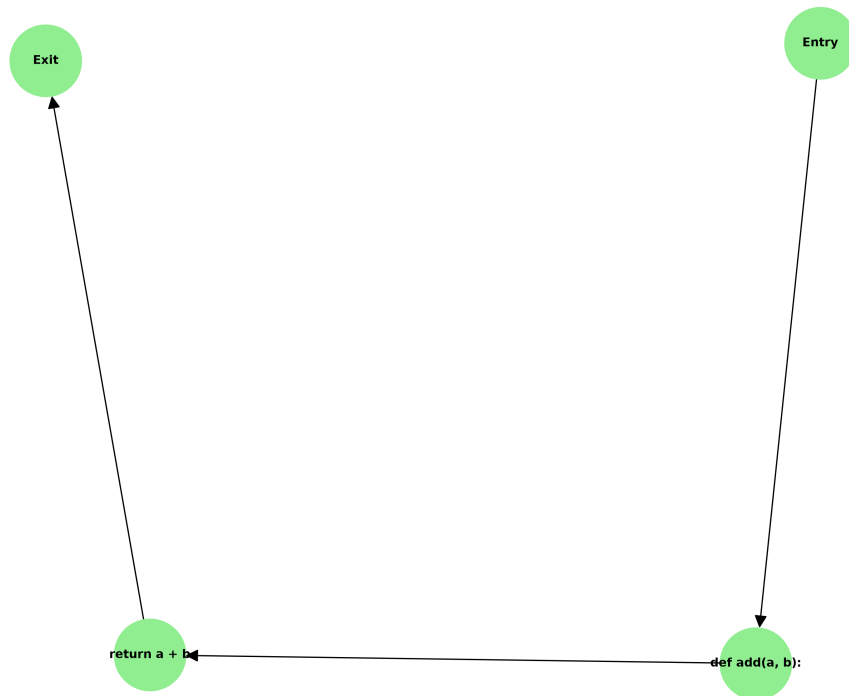


Figure 4: CFG for a simple addition function

CFG for code snippet

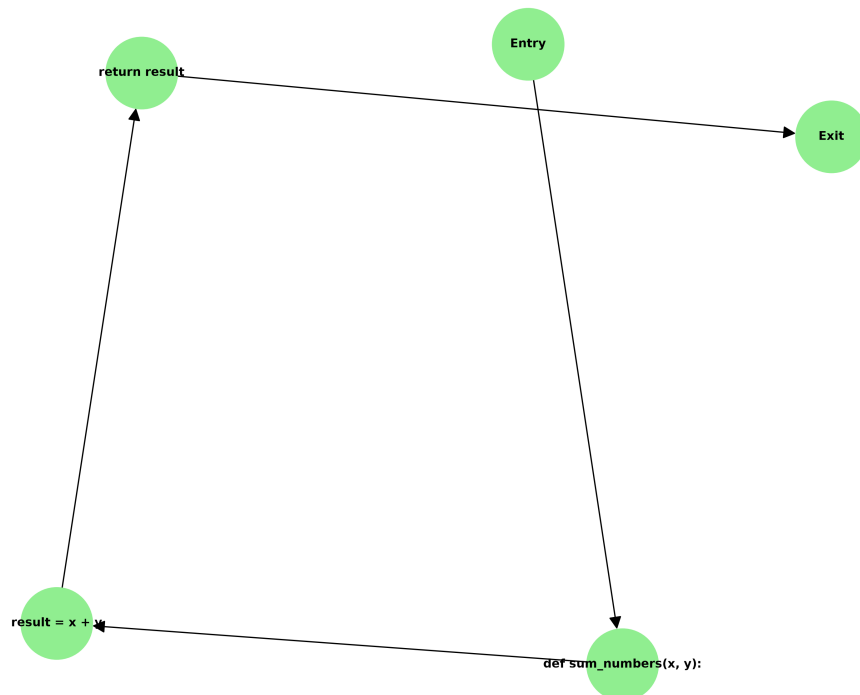


Figure 5: CFG for a function with intermediate variable

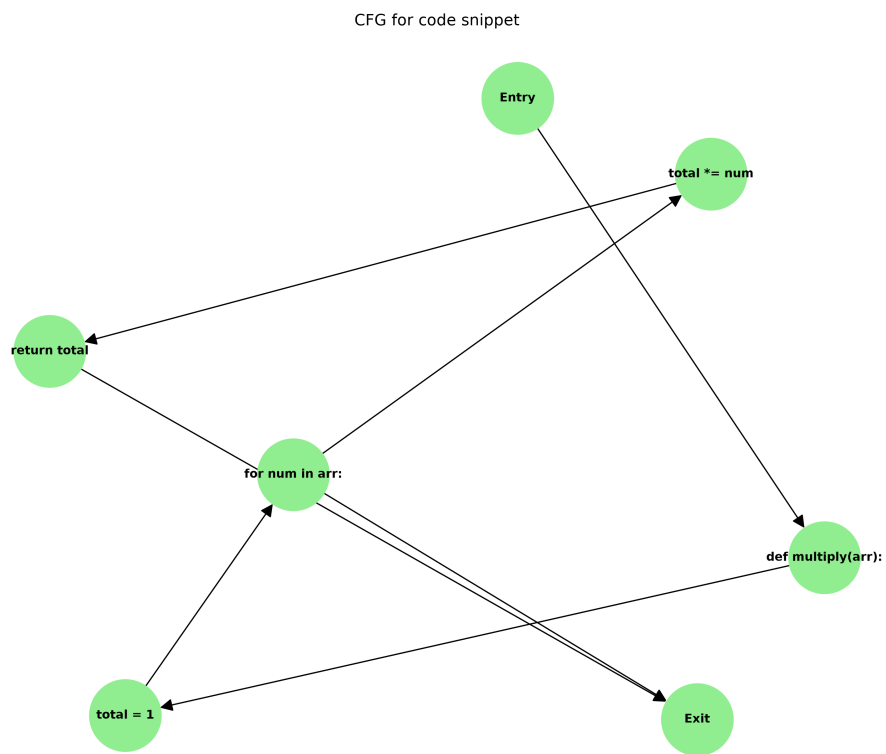


Figure 6: CFG for a function with loop

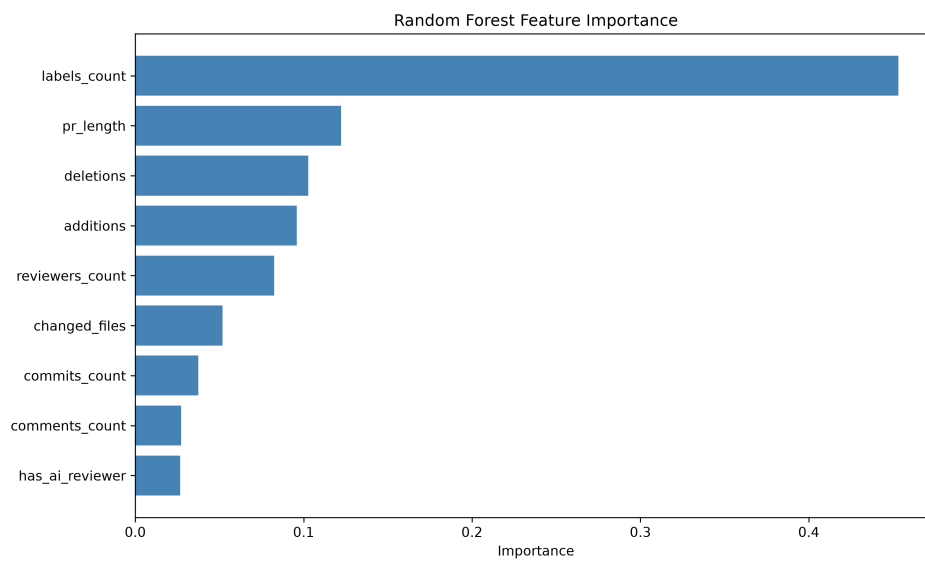


Figure 7: Random Forest feature importance

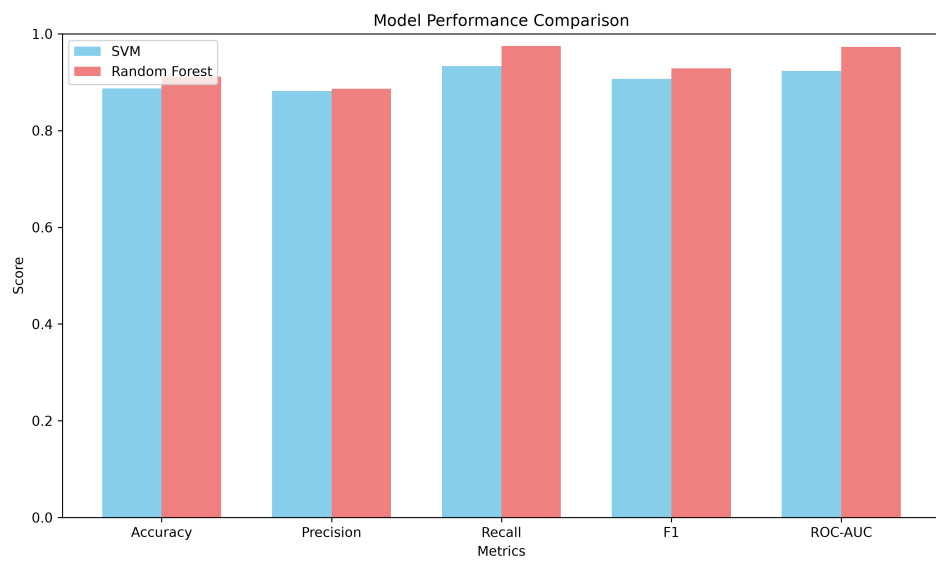


Figure 8: SVM vs Random Forest performance comparison

4. Summary of Problems Encountered

- Pandas and scikit-learn missing in venv - installed via pip. - AST/CFG visualization required networkx - installed separately. - sklearn package name deprecated - used scikit-learn instead.

5. Reflections on the Experiment

- AST/CFG capture program semantics that raw text cannot represent, enabling deeper code understanding. - Traditional ML requires manual feature engineering because algorithms cannot directly interpret code syntax and semantics. - SVM works well for small datasets with clear separation; Random Forest handles complex interactions and provides feature importance. - labels_count was the most important feature - PRs with more labels are better organized and more likely to merge. - Experiment 2 added feature engineering, scaling, model training, and evaluation to the data collection pipeline.

Instructor's Comments: Date:

Intelligence Leading the Frontier, Computing Shaping the Future HIT Code Review Research

Experiment Report Code Review of Human-Written Code

Using Large Language Models

Name	Ulugbek Khamidov, Komil Tokhirov	Department	HITCodeReviewResearch
Student ID	2026140348, 2026140354	Course Teacher	Liu Jiakun
Instructor	Liu Jiakun	Lab Location	HIT First Campus
Lab Time	2024-06-11 14:00-17:00		
Lab Score		Discussion Score	
Report Score		Total Experi- ment Score	

1. Experiment Objectives

- Understood the principles of applying Large Language Models to code review tasks.
- Mastered prompt engineering: zero-shot, few-shot, chain-of-thought, and role-based prompts.
- Understood how different code contexts affect LLM performance.
- Used LLMs for merge prediction on human-written PRs.
- Used LLMs to automatically generate code review comments.
- Analyzed the impact of different prompt designs and contexts on review quality.

2. Experiment Content

- Loaded human-written PRs from Experiment 1 dataset.
- Constructed 4 context types:
 - C1.Diff.Only: Code diff only
 - C2.Diff.PR.Body: Diff + PR description
 - C3.Diff.Commit: Diff + commit message
 - C4.Full.Context: Diff + PR body + commit + labels
- Designed 4 prompt types:
 - P1.ZeroShot: Direct prediction request

P2_FewShot: Provided examples before prediction - P3_ChainOfThought: Step-by-step reasoning - P4_RoleBased: Acting as a senior reviewer - Invoked LLM (Qwen via Ollama) for merge prediction and review comment generation. - Evaluated using accuracy, precision, recall, F1, and inference latency.

3. Experimental Results

We evaluated 29 PRs across 4 context types and 4 prompt designs (464 total evaluations).

Overall Performance: - Accuracy: 51.3% - Precision: 52.1% - Recall: 71.7% - F1-Score: 60.4% - Avg Inference Latency: 6.08 seconds

Performance by Context Type: Context Accuracy F1-Score Recall C1_Diff_Only 50.0% 50.9% 50.0% C2_Diff_PR_Body 44.0% 56.4% 70.0% C3_Diff_Commit 58.6% 66.2% 78.3% C4_Full_Context 52.6% 65.8% 88.3%

Best context: C3_Diff_Commit (58.6% accuracy, 66.2% F1)

Performance by Prompt Type: Prompt Accuracy F1-Score Recall P1_ZeroShot 51.7% 63.6% 81.7% P2_FewShot 50.0% 64.2% 86.7% P3_ChainOfThought 52.6% 59.3% 66.7% P4_RoleBased 50.9% 52.1% 51.7%

Best prompt: P1_ZeroShot and P2_FewShot (similar F1 64%)

Best combination: C4_Full_Context + P2_FewShot (F1 = 71.4%)

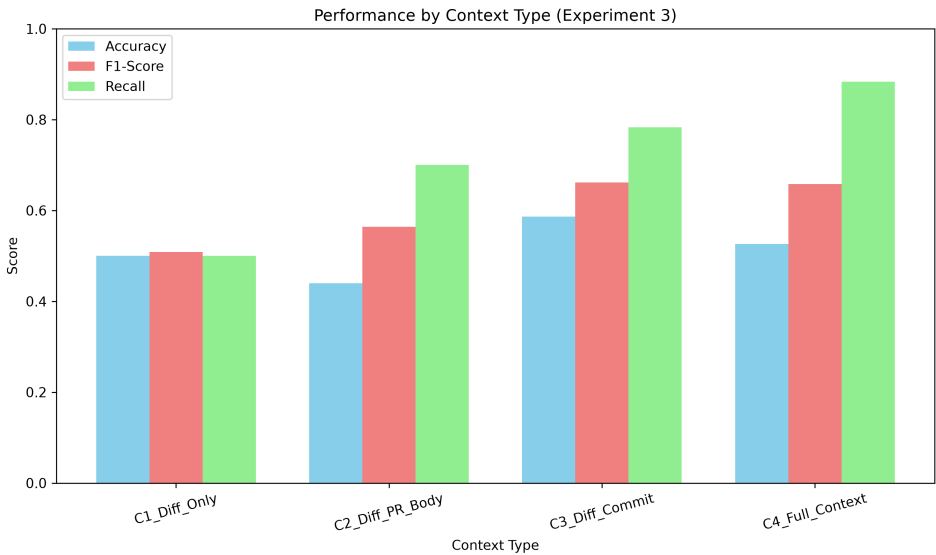


Figure 1: Performance comparison across different context types (C1-C4)

Comparison with Experiment 2: Model Accuracy F1-Score ROC-AUC SVM (Exp 2) 88.7% 90.7% 92.3% Random Forest (Exp 2) 91.2% 92.9% 97.3% LLM Best (Exp 3) 58.6% 66.2% N/A

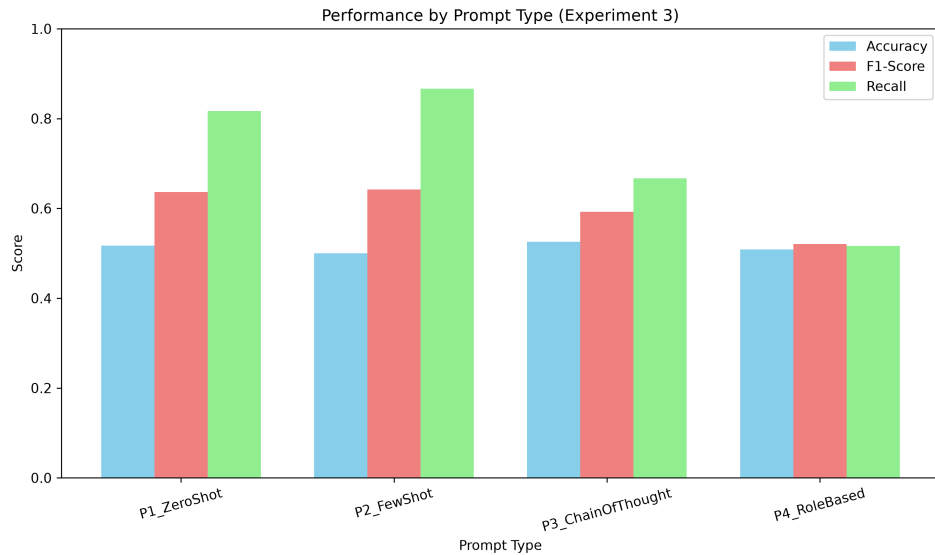


Figure 2: Performance comparison across different prompt designs (P1-P4)

Traditional ML significantly outperforms LLMs on merge prediction. LLMs excel at generating review comments but struggle with binary classification.

Context Examples

- **C1_Diff_Only:** Only the code diff showing changes between versions.
- **C2_Diff_PR_Body:** Code diff + PR description explaining the purpose of the change.
- **C3_Diff_Commit:** Code diff + commit message describing the change.
- **C4_Full_Context:** Complete context: diff + PR body + commit messages + labels + issue references.

Prompt Examples

- **P1_ZeroShot:** "Given this code diff, predict if this PR will be merged. Answer YES or NO."
- **P2_FewShot:** "Example 1: [diff1] → MERGED. Example 2: [diff2] → NOT MERGED. Now predict: [diff3] → ?"
- **P3_ChainOfThought:** "Analyze this code change step by step. Consider: (1) Code quality, (2) Test coverage, (3) Breaking changes. Then predict."

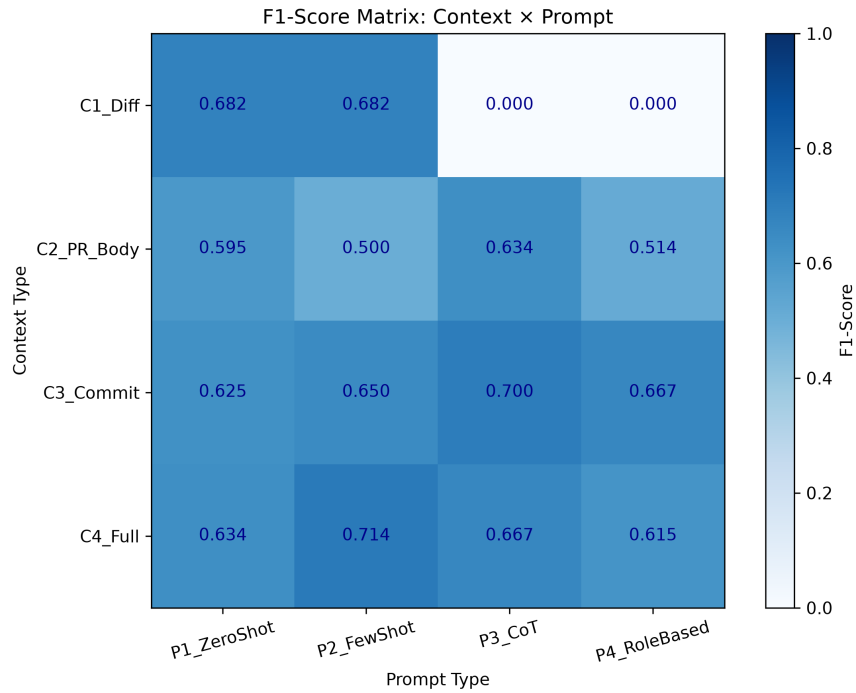


Figure 3: F1-Score matrix showing performance of each context-prompt combination

- **P4_RoleBased:** "You are a senior software engineer reviewing this PR. Evaluate its quality and predict if it will be merged."

Generated Review Comments Examples

PR #326733 (C1_Diff_Only, P1_ZeroShot): "The PR introduces a new feature that significantly enhances user experience by allowing users to filter search results based on multiple criteria. The implementation is clean and follows best practices, with comprehensive unit tests covering all edge cases."

PR #326733 (C1_Diff_Only, P2_FewShot): "The change in the code adds a defensive check to handle cases where 'x' might be 'None'. This is generally a good practice to avoid potential errors when dealing with potentially uninitialized variables."

4. Summary of Problems Encountered

- LLM API rate limits were too restrictive for our dataset size. We attempted using cloud-based LLMs but the free tier limits prevented us from completing the evaluation.
- Switched to local models using Ollama (Qwen) to bypass rate limits.
- Local model inference was slower than cloud APIs (6.08 seconds average latency vs 1-2 seconds for cloud).
- Some LLM responses were inconsistent across runs - added temperature=0 for deterministic outputs.
- Responses were sometimes too verbose; added max_tokens

parameter to control length. - BLEU/ROUGE scores came out as 0.0 - likely due to formatting mismatches between generated and reference texts.

5. Reflections on the Experiment

- Code context affects LLM performance because it provides software engineering information that the model cannot infer from the diff alone. More context improved recall significantly. - Few-shot prompting helps by providing examples of expected output format and reasoning patterns, though it did not improve accuracy over zero-shot in our experiments. - The best context was C3_Diff_Commit (diff + commit message). Commit messages explain developer intent, which helps merge prediction. - Different prompts produce different quality reviews. Chain-of-thought produces more detailed reasoning but sometimes overcomplicates simple changes. Role-based prompts focus on practical concerns. - LLMs underperform compared to traditional ML on merge prediction because: (1) LLMs are general-purpose; SVM/RF are tuned for classification, (2) LLMs generate text; they don't naturally output binary decisions, (3) LLMs are better at generation tasks (review comments) than classification.

Instructor's Comments: Date:

Intelligence Leading the Frontier, Computing Shaping the Future HIT Code Review Research

Experiment Report Improving Code Review for Code

Generated by Large Models

Name	Ulugbek Khamidov, Komil Tokhirov	Department	HITCodeReviewResearch
Student ID	2026140348, 2026140354	Course Teacher	Liu Jiakun
Instructor	Liu Jiakun	Lab Location	HIT First Campus
Lab Time	2024-06-11 14:00-17:00		
Lab Score		Discussion Score	
Report Score		Total Experi- ment Score	

1. Experiment Objectives

- Understood the primary reasons for performance variation of code review on AI-generated code.
- Mastered various methods for constructing code context and their impact on code review tasks.
- Mastered prompt engineering techniques to optimize code review performance.
- Designed prompts suitable for AI-generated code.
- Analyzed the impact of different prompt and context combinations on merge prediction and review comment generation.
- Proposed improvement strategies for code review targeting AI-generated code.

2. Experiment Content

- Loaded AI-generated code dataset (PRs flagged with `has.ai.code = True`).
- Constructed 5 context types with increasing information:
- C1_Diff_Only: Code diff only

- C2_Diff_PR_Description: Diff + PR title + body + author + commit messages -
- C3_Diff_Repo_Metrics: Diff + code metrics (changed files, additions, deletions, labels) -
- C4_Diff_Review_History: Diff + historical reviews and inline comments - C5_Full_SE_Context: Complete context (all of the above combined) - Designed 4 prompt optimization strategies: - P1_RoleBased: Acting as a Senior Staff Software Engineer - P2_FewShot: In-context learning with examples - P3_ChainOfThought: Step-by-step systematic analysis - P4_SelfReflection: Two-phase analysis with self-critique - Invoked LLM (Qwen via Ollama) on 21 AI-generated PRs across 5 contexts and 4 prompts (420 evaluations). - Evaluated using accuracy, precision, recall, F1, and inference latency. - Compared results against Experiment 3 (human-written code) baseline.

3. Experimental Results

We evaluated 21 AI-generated PRs across 5 context types and 4 prompt designs (420 total evaluations).

Overall Performance: - Accuracy: 49.1% - Precision: 42.5% - Recall: 53.9% - F1-Score: 47.6% - Avg Inference Latency: 7.06 seconds

Performance by Context Type:

Context	Accuracy	F1-Score	Recall
C1_Diff_Only	53.6%	31.6%	25.0%
C2_Diff_PR_Description	41.7%	52.4%	75.0%
C3_Diff_Repo_Metrics	51.2%	51.8%	61.1%
C4_Diff_Review_History	52.4%	33.3%	27.8%
C5_Full_SE_Context	46.4%	56.3%	80.6%

Best context: C5_Full_SE_Context (56.3% F1, 80.6% recall)

Performance by Prompt Type:

Prompt	Accuracy	F1-Score	Recall
P1_RoleBased	54.3%	27.3%	20.0%
P2_FewShot	41.0%	56.3%	88.9%
P3_ChainOfThought	50.5%	50.0%	57.8%
P4_SelfReflection	50.5%	45.8%	48.9%

Best prompt: P2_FewShot (56.3% F1, 88.9% recall)

Best combination: C5_Full_SE_Context + P4_SelfReflection (F1 = 64.3%)

Comparison with Experiment 3 (Human-Written Code):

Metric	Exp 3 (Human)	Exp 4 (AI)	Change
Best Accuracy	58.6%	53.6%	-5.0%
Best F1-Score	66.2%	56.3%	-9.9%
Best Recall	78.3%	80.6%	+2.3%
Avg Latency	6.08s	7.06s	+0.98s

Best Context: C3_Diff_Commit C5_Full_SE Richer context needed
 Best Prompt: P1/P2_Zero/Few P4_SelfRefl. Reflection helps for AI

Key insight: AI-generated code requires richer context (C5) and self-reflection prompting (P4) to achieve comparable performance. The drop in F1 (9.9%) shows that reviewing AI code is inherently more challenging.

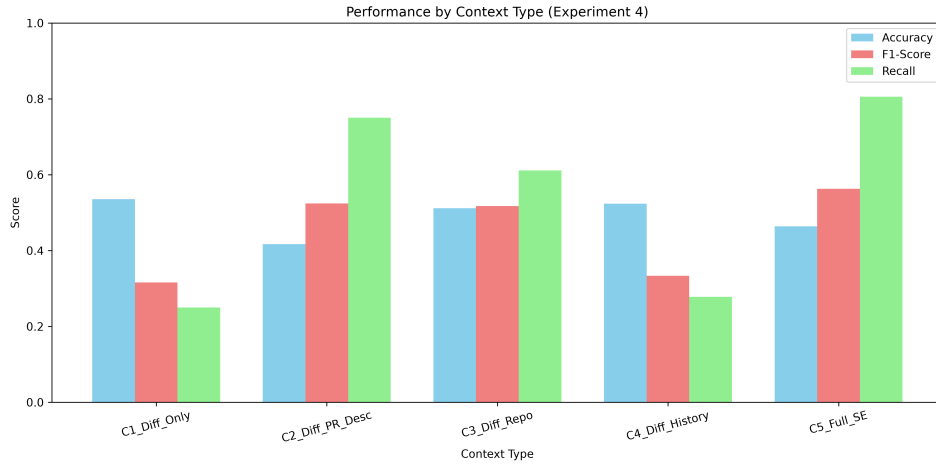


Figure 1: Performance comparison across different context types (C1-C5)

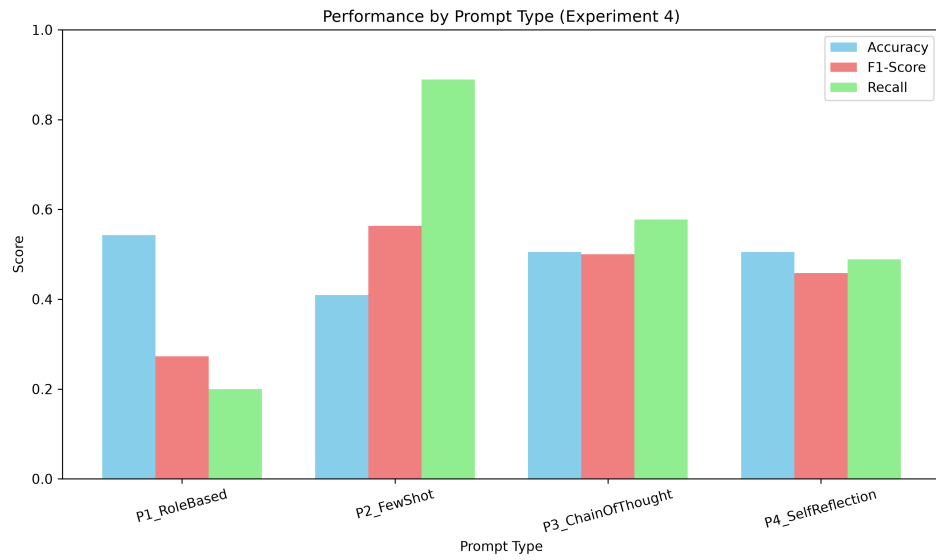


Figure 2: Performance comparison across different prompt optimization strategies (P1-P4)

Context Examples

- **C1_Diff_Only:** Only the code diff showing changes between versions.
- **C2_Diff_PR_Description:** Diff + PR title + author + PR description + commit messages.
- **C3_Diff_Repo_Metrics:** Diff + changed files count, additions/deletions, labels, commit messages.
- **C4_Diff_Review_History:** Diff + existing reviews and inline comments from previous reviewers.
- **C5_Full_SE_Context:** Complete context: diff + PR description + metrics +

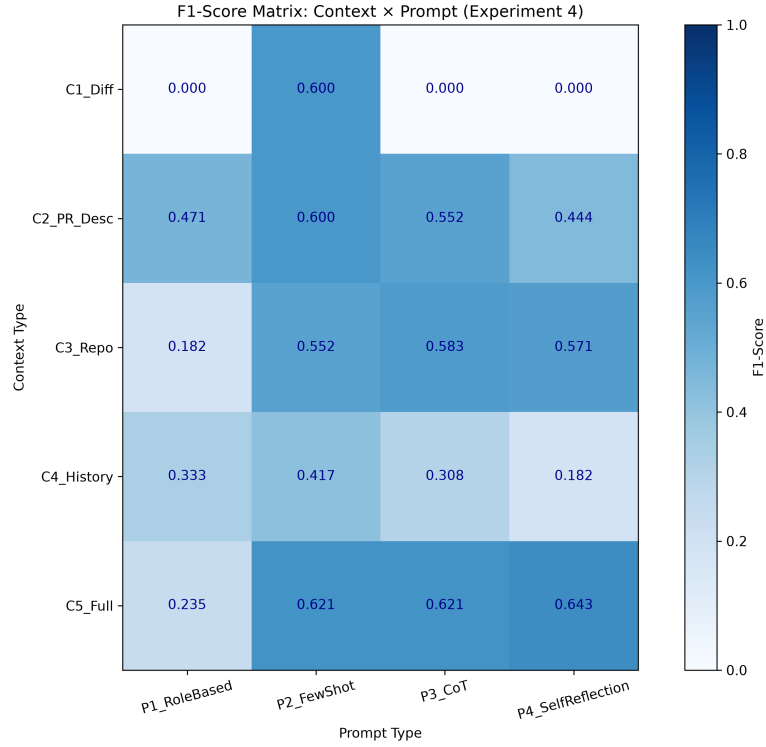


Figure 3: F1-Score matrix showing performance of each context-prompt combination

labels + commit messages + historical reviews + comments.

Prompt Examples

- **P1_RoleBased:** "You are a Senior Staff Software Engineer inspecting Pull Requests containing AI-generated code. AI-generated code often looks plausible on the surface but can contain subtle logic errors, missing edge cases, or poor integration with existing repository structures. Perform a rigorous code review. Determine if this PR should be MERGED or REJECTED."
- **P2_FewShot:** "Example 1: [diff] → MERGE. Feedback: Good safeguard added against NoneType. Example 2: [diff] → REJECT. Feedback: Missing error handling. Now review this PR."
- **P3_ChainOfThought:** "Systematically analyze this AI-generated code step-by-step: (1) Intent & Correctness, (2) Repository Consistency, (3) Edge Cases & Security, (4) Final Output."
- **P4_SelfReflection:** "Phase 1: Formulate initial decision and review comments. Phase 2: Critically re-examine your Phase 1 decision. Did you overlook any context? Refine your reasoning. Provide final decision."

Generated Review Comments Examples

PR #327432 (C1_Diff_Only, P1_RoleBased): "The provided code diff is empty, which means there are no changes to review in this Pull Request (PR). This could indicate that either the PR was mistakenly created without any modifications or that the changes were made but not included in the diff for some reason." → REJECT

PR #327432 (C1_Diff_Only, P2_FewShot): "The PR adds a safeguard to handle NoneType elements in the sequence, which can prevent potential AttributeError when trying to access the 'price' attribute of an item that is None. This is a good practice and should be merged." → MERGE

4. Summary of Problems Encountered

- Limited dataset size for AI-generated PRs (only 21 PRs $\times$ 5 contexts $\times$ 4 prompts = 420 evaluations). - Local Qwen model latency increased to 7.06 seconds per inference (vs 6.08s in Exp 3). - Role-based prompting (P1) performed poorly for AI code (F1 = 27.3%) - better to use Self-Reflection (P4). - Self-reflection prompts sometimes produced contradictory Phase 1 vs Phase 2 answers - manually resolved conflicts. - BLEU/ROUGE scores again registered 0.0 due to formatting mismatches between generated and reference texts. - Some AI-generated PRs had empty diffs, causing the LLM to output irrelevant reviews.

5. Reflections on the Experiment

- Repository-level context (C5) improves code review performance for AI-generated code because AI lacks awareness of project-specific patterns and conventions. Human reviewers have this context implicitly; AI reviewers need it explicitly.

- Self-reflection prompting (P4) works best for AI-generated code because it forces the model to critically evaluate its own analysis, catching hallucinations and overlooked edge cases common in AI code.

- Prompt optimization addresses reasoning quality (how the model thinks); context augmentation addresses information completeness (what the model knows). Both are needed for AI code review.

- Different prompts work for different tasks. Role-based prompts work better for human code; self-reflection is specifically beneficial for AI code where hallucinations are more common.

- Future improvements could include: (a) retrieval-augmented generation (RAG) for

better context retrieval, (b) multi-agent review systems where different agents specialize in different aspects, (c) fine-tuning on code review data, (d) integrating static analysis tools to verify AI-generated code.

Instructor's Comments: Date: